F1 Forecast

A cloud-native ML pipeline for Formula 1 race prediction.

F1 Forecast is an end-to-end machine learning project that aims to predict Formula 1 race outcomes and model tyre degradation and pit strategy, using data from the 2026 season. It is being built as an academic cloud computing project and is designed to be deployed on AWS.

> **Status:** Work in progress. Phase 1 (data ingestion) is implemented. The remaining phases are planned.

Why 2026 only?

The 2026 season introduced a new regulation era: revised car dimensions, a new power unit split, and the removal of DRS. Cars from earlier seasons are no longer a fair guide to current performance, so this project follows one hard rule:

- **All performance and telemetry features use 2026 data only.**
- Pre-2026 data may be used only for non-performance, contextual features (for example, historical driver standings).

Roadmap

Phase	Description	Status
1	Data ingestion (Jolpica-F1 + FastF1)	Done (Jolpica ingestion; FastF1 access tested)
2	Feature engineering	Planned
3	Tyre degradation modelling	Planned
4	Race outcome prediction (XGBoost)	Planned
5	Explainability (SHAP)	Planned
6	Streamlit dashboard	Planned
7	Backtesting	Planned

Repository contents

File	Purpose
`ingest_2026.py`	Downloads every completed 2026 round from the Jolpica-F1 API and caches raw JSON in `data/raw/`
`f1_data_pull.py`	Smoke test that confirms access to both data sources (FastF1 session + Jolpica standings)
`F1_Forecast_Synopsis.pdf`	Project synopsis document
`.gitignore`	Excludes the virtual environment and caches

What `ingest_2026.py` saves

For each completed 2026 round it stores four raw files in `data/raw/`:

- `results_round_NN.json` – finishing positions, grid positions, race status
- `qualifying_round_NN.json` – Q1 / Q2 / Q3 times
- `driver_standings_round_NN.json` – driver championship table **after** that round
- `constructor_standings_round_NN.json` – constructor championship table **after** that round

Design notes:

- **Re-runnable:** files that already exist are skipped, so nothing is downloaded twice. Run it again after each race to pick up the new round.
- **Rate-limit aware:** pauses between calls and retries on HTTP 429.
- **Leakage-aware:** standings saved for round N are the standings after race N. To predict race N, features must use standings from round N-1.

Data sources

- Jolpica-F1 API – community-maintained successor to the Ergast API (results, qualifying, standings)
- FastF1 – lap times, timing and telemetry

Getting started

Prerequisites

- Python 3.10 or newer
- Git

Setup (Windows PowerShell)

```powershell
git clone https://github.com/viyomrana/f1-forecast.git
cd f1-forecast

python -m venv f1env
.\f1env\Scripts\Activate.ps1

pip install fastf1 requests
```

On macOS / Linux, activate the environment with `source f1env/bin/activate` instead.

Run

Check that both data sources are reachable:

```powershell
python f1_data_pull.py
```

Download all completed 2026 rounds:

```powershell
python ingest_2026.py
```

Expected output: a per-round log showing `downloaded` (or `cached` on re-runs) for each of the four datasets, followed by a summary with the number of rounds and driver result rows saved.

Planned architecture (AWS)

The pipeline is designed to run on AWS using containers:

- **ECS on Fargate** – three services: a scheduled ingestion task, a long-running prediction service, and the Streamlit dashboard
- **ECR** – container images
- **S3** – raw and processed data
- **DynamoDB** – predictions and SHAP values
- **Application Load Balancer** – public entry point for the dashboard
- **EventBridge** – scheduled ingestion after each race
- **CloudWatch** – logs and monitoring
- **GitHub Actions** – CI/CD

ECS was chosen over EKS because the workload is small and Kubernetes would add complexity without benefit; Fargate was chosen over Lambda because the prediction service and dashboard are long-running.

Tech stack

Python · FastF1 · Jolpica-F1 · XGBoost · SHAP · Streamlit · Docker · AWS (ECS/Fargate, ECR, S3, DynamoDB, ALB, EventBridge, CloudWatch) · GitHub Actions

---

Author

Viyom Rana – B.Tech CSE, GLA University GitHub: @viyomrana

Acknowledgements

Thanks to the maintainers of FastF1 and the Jolpica-F1 project for making Formula 1 data openly accessible. This is an unofficial project and is not associated with Formula 1 companies.